

# Redis 缓存 三大故障

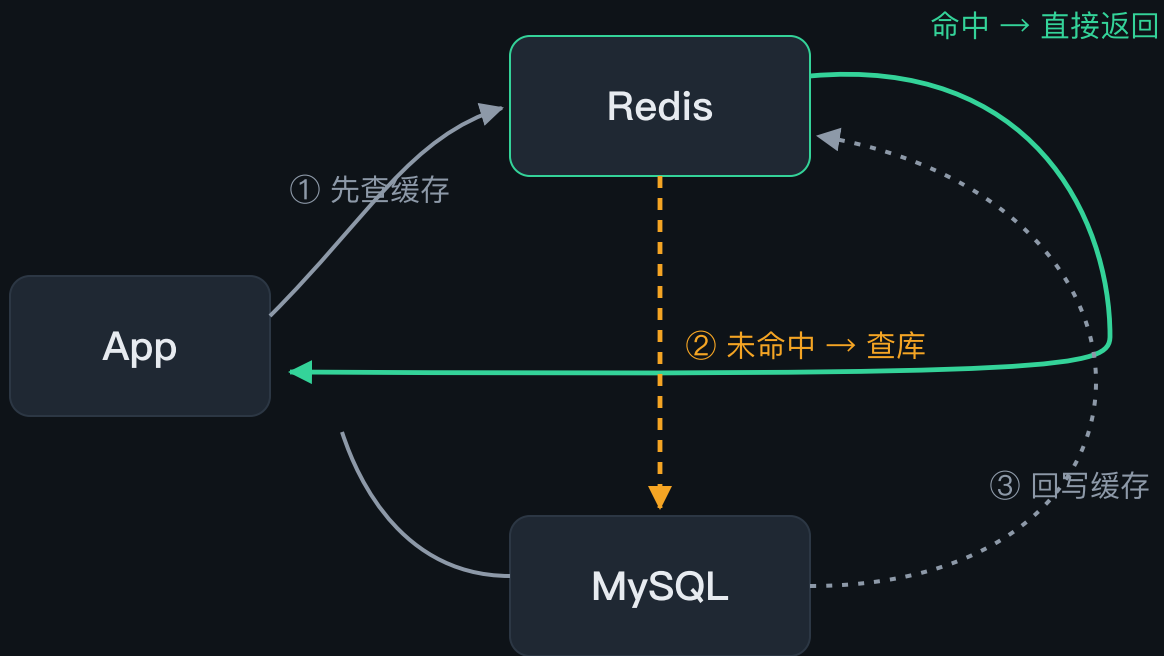
穿透 / 击穿 / 雪崩

三个名字，一个本质：

请求绕过缓存，直接压到数据库上。



# 先看正常情况：Cache-Aside 读路径



## WHY IT MATTERS

缓存能挡住绝大部分读请求。

Redis 单实例能扛**十万级** QPS，  
MySQL 单机通常只有**千到万级**。

缓存一旦“漏了”，差出来的那一两个数量级的流量，就全部落到数据库上。

穿透

漏在：数据根本不存在

击穿

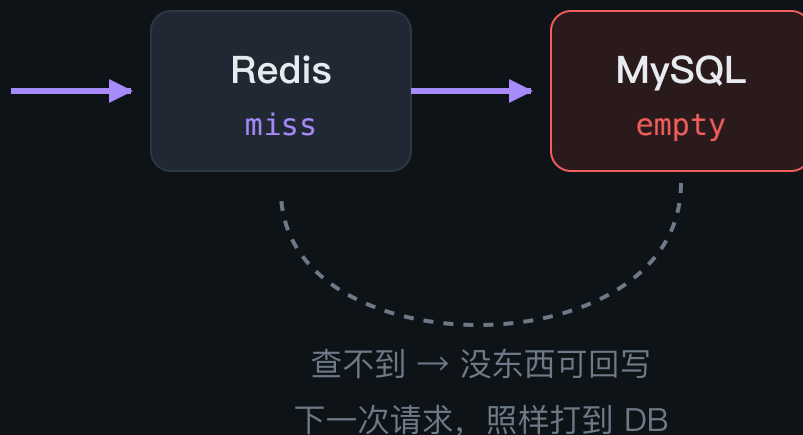
漏在：一个热点 key 过期

雪崩

漏在：一大片 key 同时失效

# 缓存穿透：查一个根本不存在的数据

```
GET /item?id=-1  
GET /item?id=-2  
GET /item?id=9e9  
..... × 10,000/s
```



## WHAT

请求的数据在缓存和数据库里都不存在。缓存永远不会被填上，每次都“穿”过去。

## WHO

恶意请求用随机 / 负数 ID 刷接口；  
爬虫遍历 ID；  
业务 bug 拼错了 key。

特征：数据不存在，key 通常很多、很分散。

# 防穿透：在请求到达 DB 之前把它拦下来

## A 参数校验

`id <= 0`、格式不对的请求，在网关或接口层直接拒绝。最便宜，但只能挡住“明显非法”的。

## B 缓存空值

DB 查不到，也往缓存写一个空值，TTL 设短一点（如 60 秒）。

代价：随机 key 多时占内存；数据新增后有短暂不一致。

## C 布隆过滤器 Bloom Filter

把所有合法 key 预先放进去，查询前先问一句。

它说“不存在”就一定不存在；说“存在”有小概率误判。

代价：标准版不支持删除，数据变化要重建或定期同步。

// B + C 合在一起的读路径 (Go)

```
func GetItem(ctx context.Context, id int64) (*Item, error) {
    if !bloom.Test(id) {                // C: 一定不存在
        return nil, ErrNotFound
    }
    v, err := rdb.Get(ctx, key(id)).Result()
    if err == nil && v == "" {           // B: 命中空值
        return nil, ErrNotFound
    }
    if err == nil {
        return decode(v)
    }
    item, err := db.FindItem(ctx, id)
    if errors.Is(err, ErrNotFound) { // B: 空值也缓存
        rdb.Set(ctx, key(id), "", 60*time.Second)
    }
    if err != nil {
        return nil, err
    }
    rdb.Set(ctx, key(id), encode(item), 30*time.Minute)
    return item, nil
}
```

# 缓存击穿：热点 key 过期的那一瞬间



## WHAT

一个高并发访问的 key 突然过期，在重建完成前，大量请求同时 miss、同时查库、同时回写。

## WHEN

秒杀商品、热搜、首页配置.....

重建越慢（多表 join、聚合计算），  
这个“窗口期”越长，打进来的请求越多。

特征：数据存在，只涉及一个或少数几个热点 key。

# 防击穿：同一时刻只让一个请求去重建

```
var g singleflight.Group

func GetProduct(ctx context.Context, id string) (*Product, error) {
    if p, err := cache.Get(ctx, id); err == nil {
        return p, nil
    }
    // 同一个 id 的并发请求在这里合并成一次
    v, err, _ := g.Do(id, func() (any, error) {
        p, err := db.QueryProduct(ctx, id)
        if err != nil {
            return nil, err
        }
        cache.Set(ctx, id, p, 10*time.Minute)
        return p, nil
    })
    if err != nil {
        return nil, err
    }
    return v.(*Product), nil
}
```

## 互斥锁重建

抢到 `SET lock NX EX 10` 的请求去查库；其余短暂等待后重读缓存。

+ 数据一致 — 有等待，锁要设超时

## 逻辑过期

key 不设 TTL，value 里带 `expire_at`。发现过期就异步重建，先返回旧值。

+ 无等待，可用性高 — 短时间读到旧数据

## 热点提前续期

识别出热点 key，在过期前由后台任务主动刷新，让它“永远不过期”。

左边的 singleflight 只在单个进程内合并：N 个实例仍会打 N 次 DB。要全局只打一次，得用 Redis 分布式锁。

# 缓存雪崩：不是一个 key，是一大片

每分钟过期的 key 数量（示意）

同一 TTL：批量预热后同时到期



TTL + 随机抖动：到期时间被摊开



```
ttl := 30*time.Minute +
    time.Duration(rand.Int63n(300))*time.Second
```

## 两种成因

- 大量 key 同时过期**  
批量写入时用了同一个 TTL
- Redis 整体不可用**  
宕机、网络分区、大 key 阻塞

## 对策

- 打散过期时间** TTL 加随机抖动（左图）
- Redis 高可用** 主从 + 哨兵，或 Redis Cluster
- 多级缓存** 进程内本地缓存兜底一层
- 保护数据库** 限流、熔断、降级返回默认值
- 缓存预热** 上线/重启前先把热数据灌进去

# 一张表分清三者

|    | 触发条件                    | 涉及的 key  | DB 里有数据? | 首选方案                              |
|----|-------------------------|----------|----------|-----------------------------------|
| 穿透 | 查询不存在的数据                | 大量、分散    | 没有       | 参数校验 + 缓存空值 / 布隆过滤器               |
| 击穿 | 热点 key 过期 + 高并发         | 一个 / 少数  | 有        | 互斥重建 (singleflight / 分布式锁) / 逻辑过期 |
| 雪崩 | 大量 key 同时过期, 或 Redis 挂了 | 一大片 / 全部 | 有        | TTL 抖动 + 高可用 + 多级缓存 + 限流降级        |

上线前自查

- ☐ 非法参数在入口拦截

☐ 热点 key 已识别, 重建有互斥

☐ Redis 有主从 / 哨兵 / 集群
- ☐ 查不到的结果也做了缓存

☐ 批量写入的 TTL 加了随机值

☐ DB 前有限流和熔断